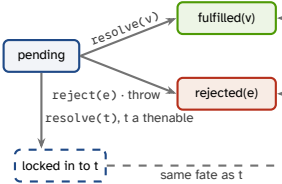


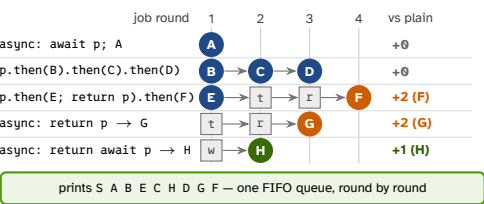
In one line: A promise is a **settle-once** result: its executor runs **now**, its handlers **always later**, each as a promise **job** (a microtask on the web). then returns a **new** promise that adopts whatever its handler returns; async/await runs on the same jobs. The work starts when it is called and a promise cannot be cancelled — an AbortSignal asks the work to stop.

1 states and fates



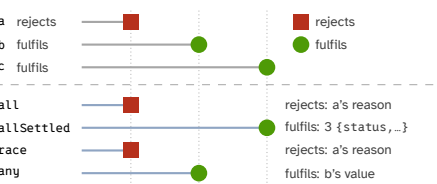
settled = fulfilled or rejected
 resolved = settled or locked in: from then on resolve, reject and a throw are ignored
 resolve(itself) → rejected with TypeError

2 counting jobs (code: left column)



t: job calling p.then : r: resolves the outer promise : w: await resumes
 returning a native promise: 2 extra jobs; return await p: 1 fewer
 since ES2019 await on a native promise takes 1 job (it took 3)

3 combinators, same inputs



no rejection: all fulfils at t3 with [a,b,c], race at t1
 all reject: any → AggregateError, .errors in input order
 nothing is cancelled: b and c run on after all rejects

Promise rules (ECMA-262)

- The **executor runs synchronously** inside new Promise; a throw in it rejects. The first resolve/reject wins; later calls do nothing.
- then never calls back synchronously: on a settled promise the job is queued at once (HTML: a microtask). A non-function argument lets the value or reason pass.
- p.then(f, r) returns a **new** promise: f returns v → fulfilled with v; throws → rejected; returns a thenable → **adopts** it. r does not see a throw from f; catch(r) = then(undefined, r) placed after it.
- Adopting thenable t: t.then is read now, called a job later; for a native promise that costs 2 rounds (drawing 2) — Stage 1 “faster promise adoption” would cut them.
- Promise.resolve(x) returns x itself when x.constructor === Promise; await uses the same check (tc39/ecma262 #1250, ES2019).
- finally(f): f gets no argument; the value or reason passes through once f's result settles; a throw or rejection in f replaces it.
- Promise.try(f, ... args) (ES2025): calls f now; a sync throw becomes a rejection. Promise.withResolvers() (ES2024): {promise, resolve, reject}.

The code of drawing 2

```

const log = (s) => console.log(s), p = Promise.resolve();
(async () => { await p; log('A'); })();
p.then(() => log('B')).then(() => log('C')).then(() => log('D'));
p.then(() => { log('E'); return p; }).then(() => log('F'));
(async () => { return p; }).then(() => log('G'));
(async () => { return await p; }).then(() => log('H'));
log('S'); // S A B E C H D G F

```

async functions

- Run synchronously up to the first await; always return a new promise: return v resolves it (a returned promise is adopted), a throw rejects it.
- await x: Promise.resolve(x), then an internal then — one job for a native promise or a plain value (await 1). Code after an await never runs in the same job; state may have changed meanwhile.
- Inside try, return await p: a plain return p leaves the try before p rejects, so catch never sees it. Top-level await (ES2022): modules only.

Rejections nobody handles

- Engine: rejecting an unhandled promise calls HostPromiseRejectionTracker(p, "reject"); a later handler calls it with "handle".
- Browser: checked after every microtask checkpoint → unhandledrejection fired from a task, cancelable (preventDefault() keeps it off the console); a handler added later → rejectionhandled.
- Node ≥ 15: --unhandled-rejections=throw by default — the 'unhandledRejection' listener if any, otherwise raised as an uncaught exception. Other modes: strict, warn, warn-with-error-code, none.

Combinators

	fulfils with	rejects with	empty input	since
all	values, input order	first rejection	[] at once	2015
allSettled	{status, value} / {status, reason}	never	[] at once	2020
race	first to settle	first to settle	never settles	2015
any	first fulfillment	AggregateError	rejects at once	2021

Inputs go through Promise.resolve, so plain values are fine. Stage 3: Promise.allKeyed({a: pa, b: pb}) → {a, b} (Await Dictionary).

Traps

- ! pa = getA(); pb = getB(); await pa; await pb — if pb rejects while await pa waits, it has no handler yet: unhandled (Node: uncaught). Use Promise.all.
- ! new Promise(async (resolve) => ...): a throw inside rejects only the executor's own, unhandled promise; the outer one stays pending.
- ! Promise.race([work, timeout]) settles, but the work and the timer run on — pass an AbortSignal instead.

Concurrency shapes

```

const a = await getA(); const b = await getB(); // tA + tB
const [a, b] = await Promise.all([getA(), getB()]); // max(tA, tB)
items.forEach(async (x) => { await save(x); }); // NOT awaited
for (const x of items) await save(x); // one at a time
await Promise.all(items.map(x => save(x))); // all at once
async function pool(items, n, work) { // at most n
  const out = []; let next = 0;
  const lane = async () => { while (next < items.length) {
    const i = next++; out[i] = await work(items[i]); } };
  await Promise.all(Array.from({ length: n }, lane));
  return out; } // next++ is safe: no job runs between read and write
const { promise, resolve, reject } = Promise.withResolvers();
ws.addEventListener('open', resolve, { once: true });
ws.addEventListener('error', reject, { once: true });
await promise; // an event as a promise

```

One failing lane rejects pool at once, yet the other lanes keep pulling items — give work a signal and abort it in catch.

Cancel with AbortSignal

```

const ac = new AbortController();
const signal = AbortSignal.any([ac.signal, AbortSignal.timeout(5000)]);
const res = await fetch(url, { signal }); // rejects with signal.reason
const sleep = (ms, signal) => new Promise((resolve, reject) => {
  signal?.throwIfAborted(); // a throw here = rejection
  const stop = () => { clearTimeout(t); reject(signal.reason); };
  const t = setTimeout(() => {
    signal?.removeEventListener('abort', stop); resolve(); }, ms);
  signal?.addEventListener('abort', stop, { once: true });
});

```

- abort(reason) runs the abort algorithms, then the abort event, synchronously. Default reason: an AbortError DOMException; timeout() uses TimeoutError; any() takes the reason of the signal that fired.
- any() cannot unsubscribe from its inputs: remove your own listeners when done. Node: await setTimeout(ms, v, { signal }) from node:timers/promises.

Retry with full jitter

```

async function retry(fn, { tries = 5, base = 100, cap = 10_000, signal } = {}) {
  for (let i = 0; i < tries; i++) {
    try { return await fn(signal); }
    catch (e) {
      if (i + 1 >= tries || signal?.aborted) throw e;
      await sleep(Math.random() * Math.min(cap, base * 2 ** i), signal);
    }
  }
}

```

Full jitter = random(0, min(cap, base·2^{attempt})) — the least work of the variants in Brooker's 2015 simulation. Retry only idempotent requests (PUT, DELETE, GET, HEAD, OPTIONS, TRACE); honour Retry-After on 429 / 503.

Async iteration and disposal

```

async function* pages(url, signal) { // Symbol.asyncIterator
  while (url) {
    const page = await (await fetch(url, { signal })).json();
    yield* page.items; url = page.next; } }
for await (const it of pages('feed')) if (it.last) break; // -> return()
const all = await Array.fromAsync(pages('feed')); // ES2026, one by one

```

- break / throw in for await calls return(): the generator's finally runs. Calls to next() queue — one at a time. In an async generator yield v awaits v first.
- for await (x of [p1, p2]) also works, one at a time — a p2 that rejects while p1 is awaited has no handler yet (the first trap).
- await using r = ... (finished, ES2027): at block exit, also on throw, r[Symbol.asyncDispose]() (else Symbol.dispose) is awaited, in reverse order; errors combine into SuppressedError. AsyncDisposableStack holds many.

Remember

Executor now, handlers later · every then is a new promise · a returned native promise costs 2 extra jobs · nothing stops unless a signal asks.